

HTR AI Brain — Backend Management Guide

A practical reference for managing the AI Analyst backend. Written for someone comfortable with a terminal but not necessarily a Python developer.

Table of Contents

1. [How the AI Works](#)

2. [Starting and Stopping the Backend](#)

3. [Adding Knowledge to the AI](#)

4. [Monitoring the AI](#)

5. [Managing Costs](#)

6. [Updating the AI's Personality / Instructions](#)

7. [Production Deployment on Railway](#)

8. [When Things Go Wrong](#)

1. How the AI Works

Plain-English Overview

The AI Analyst is not a general-purpose chatbot. It is a **specialist** — it only knows what you have explicitly given it. Everything it says is grounded in documents you control.

The underlying technique is called **RAG (Retrieval-Augmented Generation)**:

1. Every document you give the AI is broken into chunks and converted into numerical "embeddings" — a kind of coordinate in meaning-space. These are stored in Supabase (or a local file as fallback).

2. When a user asks a question, the AI converts that question into the same kind of embedding and finds the most similar document chunks.

3. Those chunks are handed to the Groq LLM (Llama 3.3 70B) along with the question and the system prompt. The model reads the retrieved chunks and writes an answer grounded in them.

4. The answer streams back to the user word-by-word.

The user's conversation history is also passed in on each request (up to ~4,096 tokens), so the AI maintains context across a session.

What the AI Knows

There are two content sources:

PDFs — Any **.pdf** file placed in **backend/data/** is parsed page-by-page and ingested. These are the primary source of deep, document-level knowledge (reports, legislation, research papers, clinical guidelines, etc.).

Sanity CMS — On every index build, the backend fetches live content from your Sanity project across eight document types:

Sanity Type	Fields indexed
policyAnalysis	title, pillar, summary, body text
post	title, body text
academyModule	title, pillar, summary, learning objectives, body
caseStudy	title, pillar, summary, body text
definition	term, description, pillars

Sanity Type	Fields indexed
analystNote	title, pillar, body text
webinar	title, pillar, description
report	title, pillar, abstract

Each Sanity document is truncated to 8,000 characters to stay within token budgets.

What the AI Cannot Do

- It cannot browse the internet or access any URL you have not given it.
- It cannot answer questions about content that has not been indexed (content published in Sanity after the last index build is invisible until you re-index).
- It cannot perform calculations, execute code, or access live data feeds.
- It does not retain memory between separate browser sessions — each conversation starts fresh from the history the frontend sends.
- It cannot answer questions reliably if your PDFs or Sanity content do not address the topic. In that case, it will say so (or, if it confabulates, see Section 8).

2. Starting and Stopping the Backend

Prerequisites

You need these installed once:

```
# Confirm Python 3.11+
python3 --version

# Install dependencies (run once, or after requirements.txt changes)
cd /Users/baba/Vermont-Health-Platform/backend
pip install -r requirements.txt
```

Ensure `backend/.env` exists and has real values (copy from `backend/.env.example`). `GOOGLE_API_KEY` is the only strictly required variable to get answers. Without Supabase vars, the index falls back to local JSON files in `backend/storage/`.

Starting the Backend

Open a dedicated terminal window for the backend. Keep it open — closing it stops the AI.

```
cd /Users/baba/Vermont-Health-Platform/backend
uvicorn main:app --reload --port 8000
```

The `--reload` flag means the server restarts automatically when you edit `main.py`. Remove `--reload` in production (Railway handles restarts differently).

How to Tell It Is Ready

Watch the logs. A clean startup with an existing index looks like this:

```
INFO      htr-brain: 🚀 HTR AI Brain v3 starting...
INFO      htr-brain: ✅ Loaded existing index from Supabase pgvector
INFO      htr-brain: Index ready. POST /api/ingest to refresh with new content.
INFO      uvicorn.error: Application startup complete.
```

Or with the local JSON fallback:

```
INFO      htr-brain: ✅ Loaded existing index from local storage/ (fallback)
INFO      htr-brain: Index ready. POST /api/ingest to refresh with new content.
```

Once you see "Application startup complete" the server accepts requests. Confirm with:

```
curl http://localhost:8000/health
```

Expected response:

```
{
  "status": "ok",
  "index_ready": true,
  "model": "llama-3.3-70b-versatile",
  "embedding_model": "gemini-embedding-001",
  "vector_store": "pgvector",
  "auth_enabled": true
}
```

If `index_ready` is `false`, the index is still building. Wait and re-check.

First-Run Startup (No Existing Index)

The first time you start with no index in Supabase and no `backend/storage/` directory, the startup log will show a full build. This is normal and takes time:

```
INFO      htr-brain: 🚀 HTR AI Brain v3 starting...
INFO      htr-brain: No existing index found – building from scratch...
INFO      htr-brain: 📄 Loading 4 PDF(s) from data/...
INFO      htr-brain: ✓ 312 pages loaded from PDFs
INFO      htr-brain: 🔗 Fetching Sanity CMS content...
INFO      htr-brain: ✓ policyAnalysis: 12 docs
INFO      htr-brain: ✓ post: 28 docs
... (one line per Sanity type)
INFO      htr-brain: ⌚ Embedding 340 documents...
... (progress bar)
INFO      htr-brain: ✅ Index built and stored in Supabase pgvector
```

Do not stop the server during this phase. Subsequent startups load from Supabase in about 2 seconds.

Stopping the Backend

Press `Ctrl+C` in the backend terminal. The server logs:

```
INFO      htr-brain: 🛑 HTR AI Brain shutting down...
```

That is a clean stop. If the process is stuck, press `Ctrl+C` twice or run:

```
# Find and kill the process on port 8000
lsof -ti:8000 | xargs kill -9
```

3. Adding Knowledge to the AI

Adding PDF Documents

1. Copy your PDF into `backend/data/`:

```
cp ~/Downloads/new-report.pdf /Users/baba/Vermont-Health-Platform/backend/data/
```

2. Trigger a re-index (see below). The AI will not see the new PDF until you do.

What PDF formats work best:

- Text-based PDFs (the kind you can select text in) index perfectly.
- Scanned PDFs (images of pages) produce garbage — no text to extract. You would need to run OCR first (e.g., Adobe Acrobat's "Make Searchable PDF" feature).
- Large PDFs (200+ pages) work fine — each page becomes a separate chunk.
- Password-protected PDFs will fail silently. Remove the password before adding.
- Avoid PDFs that are mostly charts/tables with no prose — the AI will have little to retrieve from them.

How the Sanity Content Sync Works

The backend does not watch Sanity for changes in real time. Instead, every call to `build_index()` (either at first startup or via `/api/ingest`) re-fetches all Sanity content fresh via GROQ queries over HTTPS.

This means:

- Publishing a new article in Sanity does **not** update the AI automatically.
- You must trigger a re-index after publishing significant content.
- Deleted Sanity documents will disappear from the AI on the next re-index.

Forcing a Re-Index

After adding PDFs or publishing Sanity content, trigger a re-index without restarting the server:

```
# Without INGEST_SECRET set (local dev):
curl -X POST http://localhost:8000/api/ingest

# With INGEST_SECRET set (production):
curl -X POST https://your-railway-app.railway.app/api/ingest \
  -H "Authorization: Bearer your_ingest_secret_here"
```

The endpoint returns immediately:

```
{"status": "accepted", "message": "Index rebuild started. Watch server logs for progress."}
```

The rebuild runs in the background. The existing index stays live and answers questions while the new one is being built. When the rebuild finishes, the new index swaps in atomically. Watch the server logs to confirm completion:

```
INFO      htr-brain: 🔄 /api/ingest – rebuilding index in background...
INFO      htr-brain: ✅ Background rebuild complete.
```

How Long Indexing Takes

Indexing time depends mainly on the number of documents:

Documents	Approximate time
~50	30–60 seconds
~200	2–4 minutes
~500	5–10 minutes
1000+	15+ minutes

What affects it:

- **Gemini embedding API rate limits** — the embedding model has a request-per-minute limit. LlamaIndex batches requests, but very large corpora will hit rate limits and pause with exponential backoff.
- **PDF page count** — each page is a separate embedding call.
- **Sanity document count** — each document with >20 characters of content is embedded.
- **Network latency** to Google's API.

If indexing seems stuck, check the logs for repeated "rate limit" or "429" messages. It is still working — just paused waiting for the API quota to reset.

4. Monitoring the AI

The /health Endpoint

Always-available, no auth required:

```
curl http://localhost:8000/health
# Production:
curl https://your-railway-app.railway.app/health
```

Response fields:

Field	Meaning
status	Always "ok" if the server is running
index_ready	true = AI can answer questions. false = index is still building.
model	The Groq LLM model currently in use
embedding_model	The embedding model in use
vector_store	"supabase_pgvector" = production. "local_json" = fallback mode.
auth_enabled	true if JWT verification is active. false = dev mode (no auth).

Check this endpoint from your uptime monitor (e.g., UptimeRobot, Railway healthcheck) to get alerted if the backend goes down.

Log Output to Watch For

The backend logs to stdout at INFO level. In development you see them in the terminal. In production, Railway captures them in the deployment logs tab.

Normal operation:

```
INFO  htr-brain: Chat: user=abc123 role=subscriber msg_len=124
```

One line per chat request showing user ID, their role, and message length.

Things to watch for:

Log message	What it means / action
✔ Loaded existing index from Supabase pgvector	Good — fast startup, index is live
✔ Index built and stored in Supabase pgvector	Good — fresh index complete
SUPABASE_JWT_SECRET not set – skipping JWT	You are in dev mode; auth is bypassed
Could not fetch roles for <id>	Supabase <code>user_roles</code> table query failed
Streaming error for user <id>	A single chat request failed mid-stream
Background rebuild failed	<code>/api/ingest</code> rebuild threw an error; check below
x PDF loading failed	A PDF in <code>backend/data/</code> could not be parsed
x <type> HTTP 401	Sanity API token is invalid or expired
x <type> HTTP 429	Sanity rate-limited; will retry automatically

Signs the AI is performing well:

- `/health` returns `index_ready: true`.
- Chat requests complete in 2–8 seconds for typical questions.
- No `Streaming error` lines in logs.

Signs of trouble:

- `/health` returns `index_ready: false` longer than 15 minutes after startup.
- Repeated `Streaming error` lines.
- Chat responses are empty or contain `[STREAM_ERROR]`.
- Startup takes longer than 10 minutes for a small corpus (could be API rate limit or network issue).

5. Managing Costs

API Cost Overview

The AI backend uses two separate API services:

Groq API (the chat / LLM model — `llama-3.3-70b-versatile`):

- Free tier:** 1,000+ requests/day, 6,000 tokens/minute. More than enough for a low-to-medium traffic platform.
- Paid tier:** Available if you outgrow the free tier. Pricing is per million tokens — very low cost compared to OpenAI or Anthropic.
- Cost per query on paid tier: fractions of a cent for typical 2,000–6,000 token exchanges.
- Monitor usage at: <https://console.groq.com>

Google Gemini API (the embedding model — `gemini-embedding-001`):

- Free tier:** 1,500 requests/day. Sufficient for re-indexing a typical corpus.
- Cost is incurred **only during index builds**, not during chat.
- Embeddings are stored in Supabase — re-indexing is only needed when content changes.
- Re-indexing 300 documents uses ~300–400 API calls (one per chunk), completing within the free daily quota unless the corpus is very large.
- Monitor usage at: <https://aistudio.google.com> → Manage API keys → View quota

What Drives Cost

Per-query costs (chat):

Each question to the AI sends via Groq:

1. The system prompt (~150–250 tokens)
2. Retrieved document chunks (~500–1,500 tokens)
3. Conversation history (up to 4,096 tokens)
4. The user's message (~50–500 tokens)

Total input per query: roughly 800–6,000 tokens. Output (the answer): 200–1,000 tokens. On the Groq free tier, this is \$0. On the paid tier, negligible cost per query.

Index-build costs (embedding):

Only incurred when `build_index()` runs — at first startup, after restart with no cached index, or on `/api/ingest`. Each document chunk is embedded once and stored in Supabase. Recommended cadence: re-index manually after publishing significant content, not on a timer.

Rate Limits and What Happens When You Hit Them

Groq rate limits (chat):

- Free tier: ~6,000 tokens/minute, 1,000+ requests/day.
- If hit: Groq returns a 429. The user sees a streaming error or `[STREAM_ERROR]`.
- No data is lost; the user can retry after a few seconds.
- To raise limits: upgrade to a paid Groq plan at console.groq.com.

Gemini rate limits (embeddings):

- Free tier: 1,500 requests/day, 5 requests/second.
- If hit during indexing: the backend automatically retries with exponential backoff.
- The build continues but takes longer. You will see `429` in the logs with pauses.
- For very large corpora (1,000+ chunks), consider spacing out `/api/ingest` calls across multiple days, or enable billing on Google Cloud to raise limits.

How to Reduce Cost / Usage If Needed

1. **Switch Groq model** — The `GROQ_MODEL` env var controls the chat model. `llama-3.3-70b-versatile` is an excellent balance of quality and speed. For a faster/cheaper option, try `llama-3.1-8b-instant`. Set in `backend/.env`.
2. **Reduce history length** — In `backend/main.py`, the `ChatMemoryBuffer` token limit is 4,096. Lowering it reduces input tokens per query. Edit the relevant line:

```
memory = ChatMemoryBuffer.from_defaults(token_limit=2048)
```

3. **Reduce re-indexing frequency** — Only call `/api/ingest` when you have actually published new content, not on a schedule.
4. **Limit document length** — Documents are currently truncated at 8,000 characters (line ~280 in `main.py`). Reducing this cuts embedding API calls but may lose context.

6. Updating the AI's Personality / Instructions

Where the System Prompt Lives

The system prompts are defined in `backend/main.py` starting around line 474:

```
BASE_SYSTEM_PROMPT = (
    "You are an expert AI Analyst for the Health Transformation Review (HTR). "
    "Your audience consists of healthcare executives, policy makers, and "
    "economists. "
```

```
        "Answer questions thoroughly and professionally, citing specific policies,
        data, "
        "and source documents where relevant. When referencing a document, name it
        explicitly "
        "(e.g. 'According to the Wyman Report...' or 'Vermont Act 167 states...').
        "
        "Focus on policy, economics, technology, clinical outcomes, and health
        equity."
    )

    ADVISORY_SYSTEM_PROMPT = (
        BASE_SYSTEM_PROMPT +
        " You are speaking with an ADVISORY-tier client – a senior healthcare
        leader or "
        "organizational decision-maker. Provide deeper strategic analysis,
        quantitative "
        "benchmarking, and actionable recommendations tailored to organizational
        implementation. "
        "Feel free to draw on comparative case studies and multi-state examples."
    )
```

How to Edit It

Open `backend/main.py` in any text editor. Modify the strings directly. The server reloads automatically (if running with `--reload`). If running in production without `--reload`, redeploy to apply changes.

Good things to customize:

- The AI's name and framing: change "Health Transformation Review (HTR)" to match any rebrand.
- Tone: add "Be concise — answer in 3 sentences or fewer unless asked to elaborate." or "Use bullet points for all multi-part answers."
- Topic scope: add "Do not answer questions unrelated to healthcare policy, economics, technology, clinical outcomes, or equity."
- Citation style: change the example citations to match your preferred format.

Making It More or Less Verbose

To encourage shorter answers, append to `BASE_SYSTEM_PROMPT`:

```
"Be concise. Lead with the direct answer, then provide supporting detail only
if necessary."
```

To encourage longer, more thorough answers:

```
"Be comprehensive. Structure your answers with clear sections and support each
claim with evidence from the source documents."
```

Tier-Aware Prompts

The function `get_system_prompt()` (line ~492) selects which prompt to use:

- Users with roles `advisory` or `admin` get `ADVISORY_SYSTEM_PROMPT`.
- All other subscriber+ users get `BASE_SYSTEM_PROMPT`.
- The frontend can optionally pass a `systemPrompt` field in the request body (limited to 800 characters); if present, it overrides both server-side prompts entirely.

Role hierarchy from lowest to highest: `free` → `subscriber` → `student` → `professional` → `advisory` → `admin`

Free-tier users are blocked entirely at the API level (HTTP 403).

To add a new tier (e.g., a "research" tier with its own prompt):

1. Add "research" to the `ROLE_HIERARCHY` list at line ~124 in the correct position.
2. Define a `RESEARCH_SYSTEM_PROMPT` string.
3. Update `get_system_prompt()` to check for that role.

7. Production Deployment on Railway

How Railway Keeps It Running

Railway runs the backend as a persistent process. It monitors the process and restarts it automatically if it crashes. The `Procfile` or Railway start command should be:

```
uvicorn main:app --host 0.0.0.0 --port $PORT
```

Note: no `--reload` in production. Railway injects the `$PORT` variable; do not hardcode port 8000 in the start command.

The working directory should be set to `/app/backend` (or wherever you place the backend code in the Railway repo).

Environment Variables to Set in Railway

In your Railway project → service → Variables tab, set all of these:

Variable	Where to get it
GROQ_API_KEY	Groq Console → console.groq.com → API Keys
GOOGLE_API_KEY	Google AI Studio → aistudio.google.com → API Keys
SANITY_PROJECT_ID	Sanity dashboard → project settings
SANITY_DATASET	Usually <code>production</code>
SANITY_API_TOKEN	Sanity dashboard → API → Tokens (read-only is enough)
SANITY_API_VERSION	<code>2023-10-01</code>
SUPABASE_URL	Supabase dashboard → Settings → API → Project URL
SUPABASE_SERVICE_ROLE_KEY	Supabase dashboard → Settings → API → service_role key
SUPABASE_JWT_SECRET	Supabase dashboard → Settings → API → JWT Secret
SUPABASE_DB_URL	Supabase → Settings → Database → Connection pooling URI
FRONTEND_URL	Your Vercel domain, e.g. <code>https://htr.vercel.app</code>
INGEST_SECRET	A random string you generate (keep it private)

Do not set `GROQ_MODEL` or `EMBEDDING_MODEL` unless you want to override the defaults.

To generate a secure `INGEST_SECRET`:

```
python3 -c "import secrets; print(secrets.token_urlsafe(32))"
```

Checking if It Is Running in Production

```
curl https://your-railway-app.railway.app/health
```

What to Do If It Crashes in Production

1. Open Railway → your service → Deployments tab.
2. Click the failed deployment to view crash logs.
3. Common crashes and fixes:
 - `ValueError: GOOGLE_API_KEY is required` — the env var is missing; add it in Railway.
 - `ModuleNotFoundError` — dependencies not installed; check the Railway build command includes `pip install -r requirements.txt`.
 - `OSError: [Errno 13] Permission denied: 'backend/storage/'` — Railway's filesystem may be read-only in some configurations; ensure Supabase is configured so the fallback local storage is not needed.
 - Out-of-memory kill (OOM) — see below.
4. After fixing env vars or code, click "Deploy" in Railway to redeploy.

Memory and CPU Limits to Watch For

Memory:

The index itself lives in memory during startup. With Supabase pgvector as the vector store, the in-memory footprint is small (the embeddings stay in Supabase, not RAM). With the local JSON fallback, a large index (1,000+ documents) can use 500MB–1GB RAM.

If Railway kills the process with OOM:

- Upgrade your Railway plan to get more RAM, or
- Ensure `SUPABASE_DB_URL` is set so the Supabase vector store is used instead of local JSON (far lower memory footprint).

CPU:

Index builds are CPU-intensive during the embedding phase. A build on Railway may take longer than on a local machine. This is normal. During a background rebuild (via `/api/ingest`), the AI continues answering questions — it does not stall.

Startup time:

Railway's healthcheck may time out if the backend does not respond within ~30 seconds. With Supabase as vector store, startup is ~2 seconds. With local JSON and a large index, startup can take 10–30 seconds. Increase Railway's healthcheck timeout if needed: Railway → service → Settings → Health Check Timeout.

8. When Things Go Wrong

Backend Won't Start

Symptom: Server exits immediately after launch.

Check 1 — Missing API key:

```
ValueError: GOOGLE_API_KEY is required in backend/.env
ValueError: GROQ_API_KEY is required in backend/.env
```

Fix: Add the missing key to `backend/.env`. Both `GOOGLE_API_KEY` and `GROQ_API_KEY` are required.

Check 2 — Missing `.env` file:

```
FileNotFoundError: [Errno 2] No such file or directory: '.env'
```

Fix:

```
cp /Users/baba/Vermont-Health-Platform/backend/.env.example \
  /Users/baba/Vermont-Health-Platform/backend/.env
# Then edit .env with real values
```

Check 3 — Port already in use:

```
ERROR: [Errno 48] Address already in use
```

Fix: Something is already on port 8000. Find and stop it:

```
lsof -ti:8000 | xargs kill -9
```

Then restart uvicorn.

Check 4 — Missing Python package:

```
ModuleNotFoundError: No module named 'llama_index'
```

Fix:

```
cd /Users/baba/Vermont-Health-Platform/backend
pip install -r requirements.txt
```

Indexing Fails Partway Through

Symptom: You see `Background rebuild failed` or the startup log stops mid-index.

Check 1 — Sanity API token invalid:

```
x policyAnalysis HTTP 401
```

Fix: Generate a new read-only token in Sanity dashboard → API → Tokens. Update `backend/.env`.

Check 2 — Embedding API quota exhausted:

```
RuntimeError: Gemini embedding quota exceeded
```

Fix: Wait for the quota to reset (Gemini free tier resets daily). The backend uses exponential backoff automatically, but if the full daily quota is gone it will fail. Re-run `/api/ingest` the next day, or enable billing on your Google Cloud project.

Check 3 — Corrupt PDF:

```
x PDF loading failed: cannot identify image file
```

Fix: Remove the offending PDF from `backend/data/`. Use `pypdf` to verify:

```
python3 -c "import pypdf; r = pypdf.PdfReader('backend/data/yourfile.pdf');  
print(len(r.pages), 'pages')"
```

If it throws an error, the file is corrupt or scanned-only.

Check 4 — Supabase DB URL wrong:

```
Could not init Supabase vector store: ... falling back to local storage
```

This is a warning, not a fatal error. The index will build to local JSON instead. To fix: copy the exact connection string from Supabase → Settings → Database → Connection pooling (use the "Transaction" mode URI).

AI Gives Wrong Answers

This is almost always a content problem, not a code problem.

The AI says it does not know about X, but you have a PDF about X:

- Confirm the PDF is in `backend/data/` and is text-based (not scanned).
- Confirm you re-indexed after adding it (`POST /api/ingest`).
- Confirm the rebuild completed (watch logs for "Background rebuild complete").

The AI gives an answer that contradicts your documents (confabulation):

- The LLM is generating text that sounds plausible but is not grounded in your documents.
- This typically happens when the retrieved chunks do not contain enough relevant context.
- Mitigation: add more content to `backend/data/` covering that topic more thoroughly.
- You can also make the system prompt more restrictive: add "If you cannot find the answer in the provided source documents, say 'I don't have information on that in my current knowledge base' — do not speculate."

The AI ignores conversation history:

- History is passed by the frontend on each request. If history is missing, check `frontend/app/api/chat/route.ts` to confirm it forwards the `history` array correctly.

AI Is Too Slow

Typical response time for a well-functioning setup: 1–5 seconds to first token, 3–12 seconds total for a typical answer.

If responses take 20+ seconds:

Check 1 — Index not ready:

```
curl http://localhost:8000/health
```

If `index_ready` is `false`, the server is still building the index. Wait.

Check 2 — Groq API latency or rate limits:

Groq occasionally has elevated latency or may throttle requests on the free tier. Check <https://status.groq.com> for incidents. On free tier, if you are hitting token/minute limits, responses will queue and appear slow. Upgrade to a paid Groq plan to raise limits.

Check 3 — Large conversation history:

Very long conversations (20+ turns) send thousands of tokens of history on every request. The `ChatMemoryBuffer` token limit is 4,096 — if conversations routinely exceed this, older messages are dropped automatically. This is expected behavior.

Check 4 — Railway server under-resourced:

If the backend is on a small Railway plan, CPU contention can slow API proxy calls. Upgrade the plan or ensure no other services share the same Railway service.

Quick Reference

Key File Paths

Path	Purpose
<code>/Users/baba/Vermont-Health-Platform/backend/main.py</code>	The entire backend — start here for any changes
<code>/Users/baba/Vermont-Health-Platform/backend/.env</code>	Secret keys and configuration
<code>/Users/baba/Vermont-Health-Platform/backend/.env.example</code>	Template for .env
<code>/Users/baba/Vermont-Health-Platform/backend/data/</code>	Drop PDFs here to add knowledge
<code>/Users/baba/Vermont-Health-Platform/backend/storage/</code>	Local JSON index fallback (gitignored)
<code>/Users/baba/Vermont-Health-Platform/backend/requirements.txt</code>	Python dependencies

Key URLs (Local Dev)

URL	Purpose
<code>http://localhost:8000/health</code>	Health check — always available, no auth
<code>http://localhost:8000/api/chat</code>	Chat endpoint — requires Bearer JWT
<code>http://localhost:8000/api/ingest</code>	Trigger re-index — requires INGEST_SECRET if set
<code>http://localhost:8000/docs</code>	FastAPI auto-generated API docs (dev only)

Common Commands

```
# Start the backend
cd /Users/baba/Vermont-Health-Platform/backend && uvicorn main:app --reload --port 8000

# Health check
curl http://localhost:8000/health

# Trigger re-index (no secret)
curl -X POST http://localhost:8000/api/ingest

# Trigger re-index (with INGEST_SECRET)
curl -X POST http://localhost:8000/api/ingest -H "Authorization: Bearer YOUR_SECRET"
```

```
# Install/update dependencies
cd /Users/baba/Vermont-Health-Platform/backend && pip install -r
requirements.txt

# Kill process on port 8000
lsof -ti:8000 | xargs kill -9

# Test a PDF file is readable
python3 -c "import pypdf; r = pypdf.PdfReader('backend/data/YOURFILE.pdf');
print(len(r.pages), 'pages')"
```